

.NET BACKEND DEVELOPMENT

Self-Study Progress Report

Prepared by	Dereje Abteu
Position	Application Management Officer (Trainee Banker)
Report Date	April 27, 2026
Study Period	April 2026 – Ongoing
Learning Resource	Julio Casal – .NET REST API Tutorial (YouTube)
Development Tools	Visual Studio Code .NET SDK v10 SQLite EF Core

1. Introduction

I am Dereje Abteu, a trainee banker currently holding the position of Application Management Officer. In addition to my primary banking responsibilities, I have been pursuing self-directed technical training in backend software development using Microsoft's .NET platform. This report documents the progress I have made during my .NET learning journey, covering the key concepts I have studied, the practical application I have built, and the challenges I have encountered along the way.

The primary objective of this training is to build a foundational understanding of RESTful API development using .NET, which is directly relevant to my role in application management. My study follows the structured tutorial series by Julio Casal on YouTube, complemented by hands-on practice using Visual Studio Code and the .NET SDK.

2. Development Environment & Tools

The following tools and technologies were set up and used throughout this study:

- Visual Studio Code (VS Code) — Primary Integrated Development Environment (IDE)
- .NET SDK Version 10 (latest) — Backend framework and runtime

- REST Client Extension for VS Code — Used to test HTTP endpoints via .http files
- NuGet Package Manager — Used to install third-party library packages
- Microsoft.EntityFrameworkCore.Sqlite — Database provider for SQLite
- Microsoft.EntityFrameworkCore.Design — Required for EF Core migration tooling
- SQLite — Lightweight relational database used for local development

3. Project Creation & Initial Setup

I initiated the project by creating a new .NET application using Visual Studio Code. The steps I followed are summarized below:

- Opened VS Code and navigated to View → Command Palette (Ctrl+Shift+P).
- Selected .NET: New Project, then chose ASP.NET Core Empty as the template. (As I advance, I intend to use ASP.NET Core Web API template directly.)
- Named the application GameStore.Api and selected the modern Solution format (.slnx) to organize the project.
- Explored the automatically generated project files to understand the scaffolded structure.
- Built the project using Solution Explorer (right-click → Build), which generated the GameStore.Api.dll output file inside the bin directory.
- Verified the build output by inspecting the bin folder in File Explorer.
- Ran the application using F5 (Fn+F5 on my machine), selecting C# as the language and HTTP as the profile.
- Alternatively used Ctrl+F5 or the terminal command dotnet run to launch the application.
- Disabled the automatic browser launch by setting launchBrowser: false inside launchSettings.json to avoid multiple browser windows opening on each run.

4. REST API Fundamentals

I studied the core concepts of REST (Representational State Transfer) and understood how HTTP verbs map to CRUD (Create, Read, Update, Delete) operations. The table below summarizes this mapping:

HTTP Method	CRUD Operation	Description
POST	Create	Adds a new resource to the server
GET	Read	Retrieves an existing resource or list
PUT	Update	Modifies an existing resource by ID
DELETE	Delete	Removes an existing resource by ID

To test these endpoints, I installed the REST Client extension in VS Code, created a games.http file within the GameStore.Api folder, and wrote HTTP request definitions. VS Code automatically

displayed a Send Request button above each request, which I used to call my running application and inspect the results.

5. Data Transfer Objects (DTOs)

One of the important concepts I studied was the use of Data Transfer Objects (DTOs). A DTO is a structured package used to control exactly what data is sent to or received from the client (frontend). This is particularly important from a security perspective: rather than exposing every field in a database record, DTOs allow us to selectively expose only the fields that are safe and necessary.

For example, if a database table contains a user's name, email address, and password hash, a DTO for the public-facing API would include only the name and email — protecting sensitive data from being inadvertently exposed.

In my project, I created the following DTOs under a dedicated Dtos folder inside GameStore.Api:

- GameDto.cs — Used to return game data to the client (read operations).
- CreateGameDto.cs — Used to receive game data from the client when creating a new game (POST operation).

6. Endpoint Implementation

I implemented all four CRUD endpoints inside Program.cs and later refactored them into a dedicated GameEndpoints.cs class for cleaner, more maintainable code. The following endpoints were implemented and tested successfully:

- GET /games — Retrieves the full list of games; returns HTTP 200 OK.
- GET /games/{id} — Retrieves a single game by its unique identifier.
- POST /games — Creates a new game entry; returns HTTP 201 Created with the location header.
- PUT /games/{id} — Updates an existing game record; returns HTTP 204 No Content upon success.
- DELETE /games/{id} — Removes a game from the list; returns the appropriate HTTP response.

I also implemented handling for non-existent resources. Initially, a GET request for an invalid ID returned HTTP 200 OK with a null body, which is incorrect behavior. After applying existence checks, the API correctly returns HTTP 404 Not Found for missing resources.

7. Code Refactoring & Clean Architecture

As the number of endpoints grew, Program.cs became lengthy and difficult to manage. I refactored the code by:

- Creating an Endpoints folder under GameStore.Api using Solution Explorer.

- Creating a static class named `GameEndpoints.cs` within that folder to consolidate all game-related API logic.
- Moving all API route definitions from `Program.cs` into `GameEndpoints.cs`, keeping `Program.cs` clean and focused on application configuration.
- Calling `MapGameEndpoints()` from `Program.cs` to register all endpoints in a single, readable line.

This approach follows the principle of separation of concerns, which is a foundational concept in professional software development.

8. Input Validation

To ensure data integrity, I implemented input validation on the POST endpoint. Without validation, it was possible to create a game entry with an empty name, which is not acceptable behavior. I applied validation using two mechanisms:

- `[Required]` attribute — Marks the name field as mandatory in the `CreateGameDto` class. This is imported using `Ctrl+.` in VS Code.
- `StringLength` attribute — Limits the maximum allowable length of the name field to prevent oversized inputs.
- Builder-level validation — Added validation middleware in `Program.cs` to ensure that all incoming request bodies are validated against these constraints before the handler is invoked.

With these measures in place, any attempt to submit a POST request with an empty name field will return an HTTP 400 Bad Request response, along with a descriptive error message.

9. Data Modeling & Database Integration

9.1 Domain Models

I created a `Models` folder under `GameStore.Api` and defined two C# model classes to represent the application's data structure:

- `Game.cs` — Contains properties such as `Id`, `Name`, `Genre` (a reference to the `Genre` model), `Price`, and `ReleaseDate`.
- `Genre.cs` — Contains `Id` and `Name` properties.

By referencing the `Genre` model inside the `Game` model, I established a relationship between the two entities, which mirrors a foreign key relationship in a relational database.

9.2 NuGet Package Installation

To enable database connectivity, I installed the following NuGet packages via Solution Explorer → Add NuGet Package:

- `Microsoft.EntityFrameworkCore.Sqlite (v10.0.7)` — Provides the SQLite database driver for Entity Framework Core.
- `Microsoft.EntityFrameworkCore.Design` — Required for running EF Core migration commands from the terminal.

NuGet is the standard package manager for .NET projects, analogous to npm in JavaScript or pip in Python. It provides access to thousands of pre-built libraries created by Microsoft and the developer community.

9.3 Entity Framework Core (EF Core) & O/RM

Entity Framework Core serves as the Object-Relational Mapper (O/RM) between my C# application and the SQLite database. In simple terms, EF Core acts as a translator: my application works with C# objects and classes, while the database works with tables and rows. EF Core handles the conversion between these two representations automatically, allowing me to interact with the database using familiar C# syntax rather than raw SQL.

I created a Data folder under GameStore.Api and defined a GameStoreContext.cs file, which inherits from DbContext. This context class exposes two DbSet properties — one for Games and one for Genres — which EF Core uses to generate and execute the corresponding database queries.

10. Application Architecture Diagram

The diagram below illustrates the layered architecture of the GameStore.Api application as built during this study:

CLIENT (HTTP Requests via REST Client Extension)		
API ENDPOINTS	—	GameEndpoints.cs (GET / POST / PUT / DELETE)
DTOS	—	GameDto CreateGameDto (Data Transfer Objects)
MODELS	—	Game Genre (C# Domain Models)
EF CORE O/RM	—	GameStoreContext (Object-Relational Mapper)
SQLite DATABASE — (via EF Core Migrations)		

Each layer has a clearly defined responsibility, ensuring that the application is modular, testable, and maintainable.

11. Database Migration

After defining the models and the database context, I proceeded to configure database migration using the EF Core CLI tooling:

- Installed the dotnet ef global tool to enable migration commands in the terminal.
- Installed the Microsoft.EntityFrameworkCore.Design NuGet package, which is required by the migration tooling.

- Ran the command: `dotnet ef migrations add InitialCreate --output-dir Data/Migrations` — This generated the migration files that define how to create the Games and Genres tables in the database. The generated migration script can be reviewed inside the InitialCreate.cs file.
- Ran the command: `dotnet ef database update` — This applies the migration to the SQLite database, physically creating the tables.

Issue Encountered & Resolution

When executing `dotnet ef database update`, I encountered an error caused by a syntax mistake in the database connection string defined in Program.cs. The connection string format for SQLite was incorrect, which prevented EF Core from locating or creating the database file. This issue has been identified and will be corrected in the next study session by reviewing the proper SQLite connection string format.

12. Overall Progress Summary

The table below provides a consolidated view of all topics covered and their current completion status:

Topic / Module	Key Achievement	Status
Project Setup & Environment	Created GameStore.Api project, built and run successfully	✓ Completed
REST API Concepts	Understood HTTP verbs and CRUD mapping	✓ Completed
REST Client Testing	Tested all endpoints via .http files in VS Code	✓ Completed
Data Transfer Objects (DTOs)	Created GameDto and CreateGameDto	✓ Completed
GET Endpoint	List and single-item retrieval with 200/404 handling	✓ Completed
POST Endpoint	New game creation returning 201 Created	✓ Completed
PUT Endpoint	Update existing game returning 204 No Content	✓ Completed
DELETE Endpoint	Delete game with correct HTTP response	✓ Completed
404 Handling	Non-existent resource returns 404 Not Found	✓ Completed
Code Refactoring	Endpoints moved to GameEndpoints.cs static class	✓ Completed
Input Validation	[Required] and StringLength	✓ Completed

Topic / Module	Key Achievement	Status
	attributes applied	
Domain Models	Game and Genre models with relationship defined	✓ Completed
NuGet Packages	EF Core Sqlite and Design packages installed	✓ Completed
EF Core / O/RM	GameStoreContext created with DbSets	✓ Completed
Database Migration	InitialCreate migration generated; update pending fix	⚠ In Progress

13. Next Steps

The following topics are planned for the upcoming study sessions:

- Resolve the SQLite connection string error and successfully apply the database migration.
- Seed the database with initial data for Games and Genres.
- Replace the in-memory list with actual database-backed queries using EF Core.
- Explore more advanced topics such as error handling middleware, async programming with async/await, and API versioning.
- Transition to using the ASP.NET Core Web API template for future projects.

14. Conclusion

This self-study journey in .NET backend development has been highly productive and directly relevant to my professional responsibilities as an Application Management Officer. Within a short period, I have successfully built a functional REST API from scratch, applied industry-standard practices such as DTO design, code refactoring, and input validation, and begun integrating a relational database using Entity Framework Core.

I am committed to continuing this training and deepening my technical knowledge, with the goal of contributing more effectively to the application management and software development functions within my organization. I look forward to reporting further progress in subsequent sessions.

Dereje Abteu

Application Management Officer (Trainee Banker)

April 27, 2026